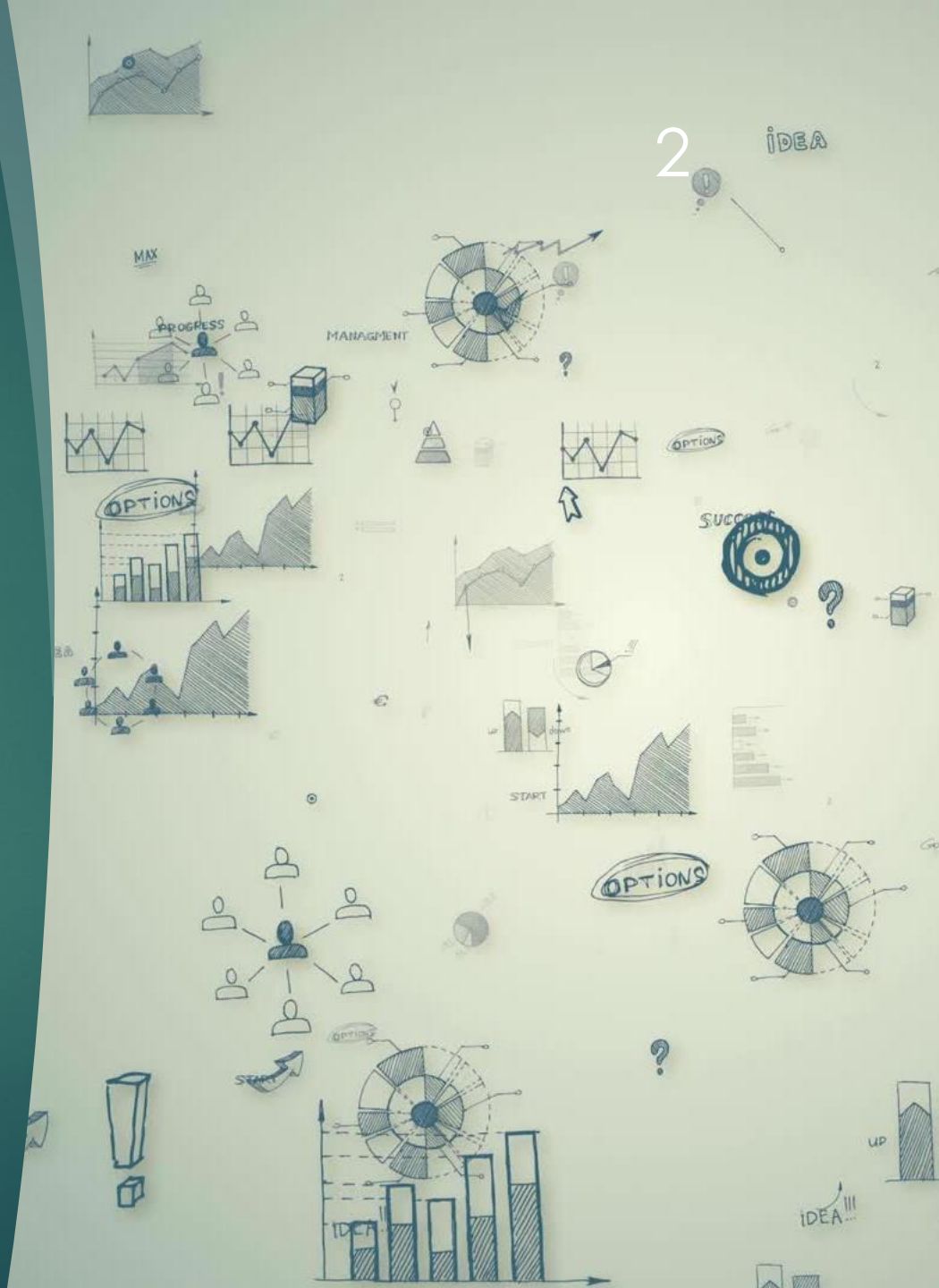


Parameter free methods

CHAPTER 5

Chapter 5 Roadmap

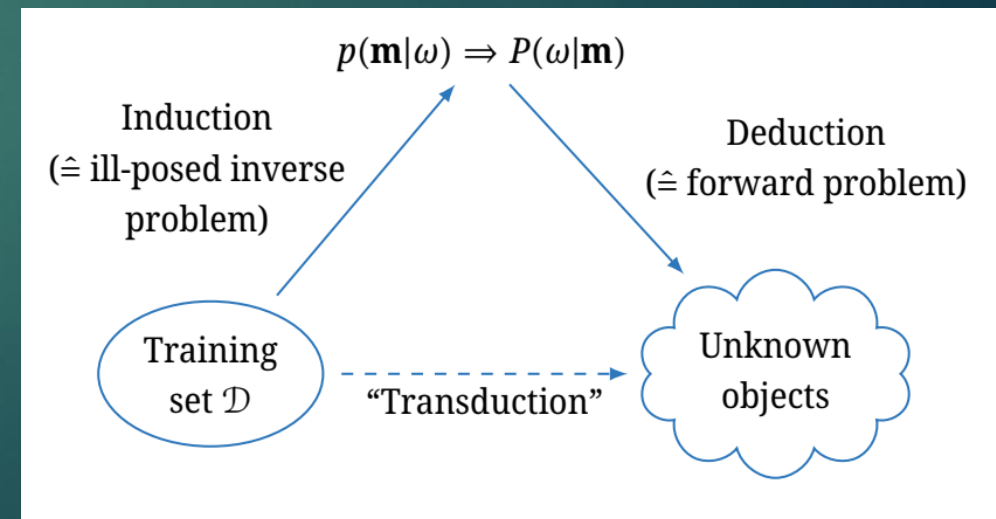
- Introduction
- Histogram Method
- Parzen-window Method (KDE)
- K-Nearest Neighbor Estimation
- K-Nearest Neighbor Classifier



The Triangle of Inference

3

- **The Core Goal: Learning Rules from Data**
 - When teaching a computer to classify objects based on their features, the process usually happens in two steps:
 1. **Induction (Going from Specific to General):** You look at a specific dataset and try to figure out a general rule or pattern (e.g., discovering how features are distributed for a certain class).
 2. **Deduction (Going from General to Specific):** You take that general rule and apply it to a brand-new, unknown object to predict its class.
- **Shortcut (Transduction):** *Instead of making a general rule first, some methods skip straight from old data to new data. However, this chapter focuses strictly on the **induction** step—building a model from data.*



What is an Inverse Problem?

4

- Finding a general rule from limited data points is an **"ill-posed inverse problem."**
- To understand what this means, consider the difference between going forward and going backward:
 1. **The Forward Problem (Easy / Deduction):** If you have a perfect 3D model of a person and a strong light source, you can easily calculate exactly what their shadow will look like on a flat wall.
 2. **The Inverse Problem (Hard / Induction):** If you are shown a blurry, 2D shadow on a wall and asked to reconstruct the exact 3D face of the person who cast it, you are working backward.
- It is called **"ill-posed"** because a single shadow could belong to many different people, or even a strangely shaped piece of cardboard. Your data (the shadow) simply doesn't contain enough information by itself to guarantee one true answer.

What is an Inverse Problem?

5

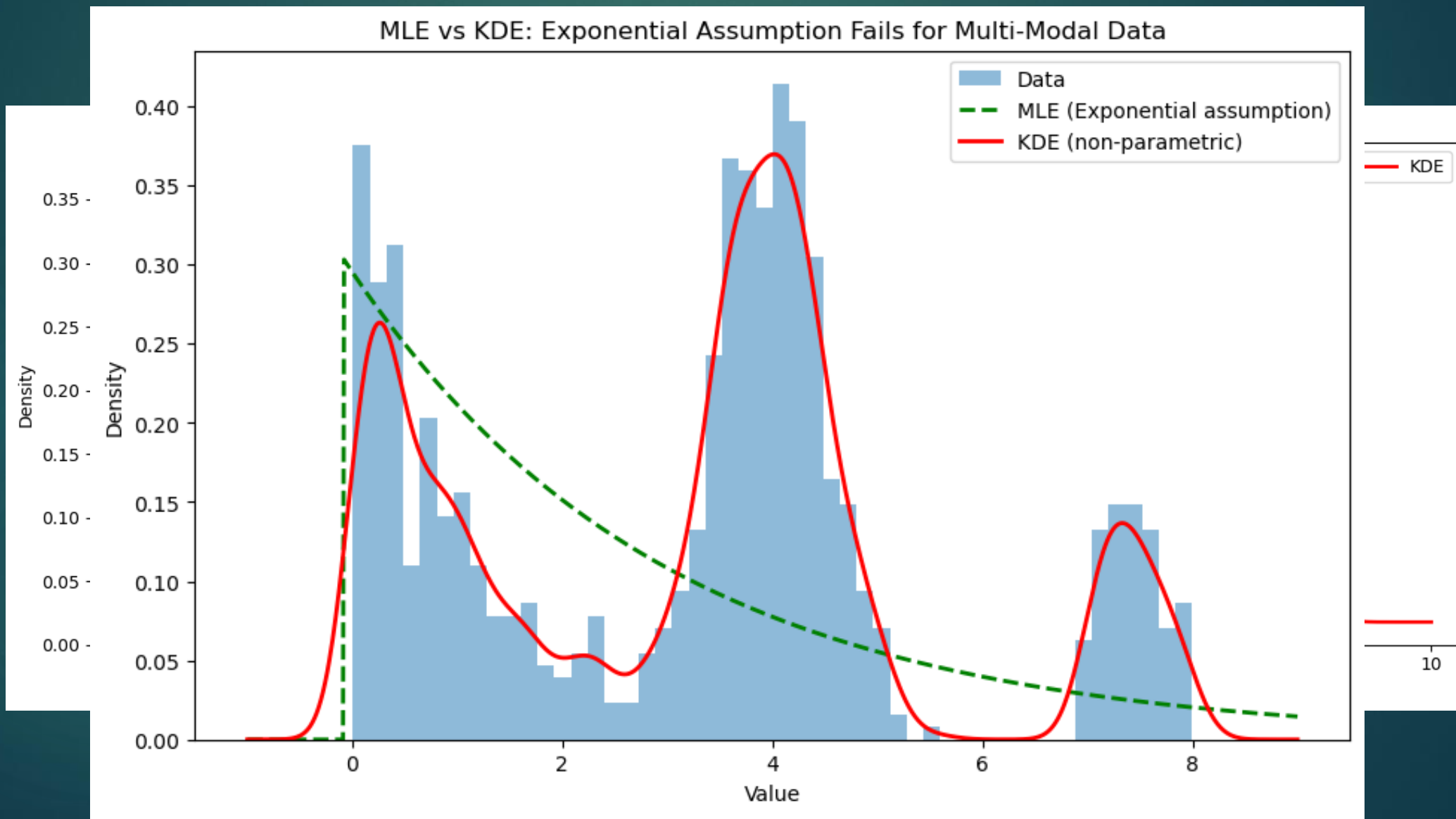
- To solve this, scientists use **regularization**.
- Think of regularization as introducing a smart assumption to guide your guess—for example, assuming that the shadow is cast by a living human being rather than a cardboard cutout.
- This narrows down the possibilities so a solution becomes findable.

Parametric vs. Parameter-Free Methods

- **Chapter 04 (Parametric):**
 - We assume the data perfectly fits a standard mathematical shape (like a Bell Curve/Gaussian distribution).
 - The problem? Real-world data rarely fits into these neat, simple shapes.
- **Chapter 05 (Parameter-Free / Non-Parametric):**
 - We don't assume any shape at all.
 - Instead, we let the data speak for itself.
 - We treat the data points as a rough sketch of the real distribution, knowing that as we gather more data, the sketch will turn into a perfect picture.

Parametric vs. Parameter-Free Methods

7



How to Estimate Probability with Data

8

- To find the probability density at any specific point, you draw a small "box" or "neighborhood" around that point and look at three things:
 - **N**: The total number of data points you have.
 - **V**: The volume (size) of the neighborhood box.
 - **k**: How many data points actually fell inside that box.
- The formula boils down to:

$$\text{Estimated Density} \approx \frac{K}{N \times V}$$

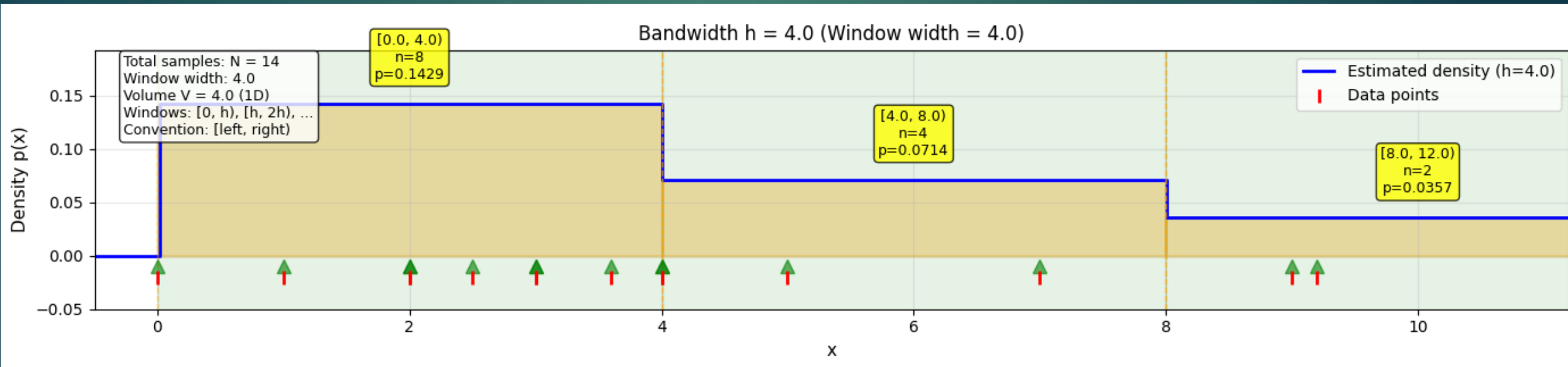
- *(Think of it like population density: Count the people in a room [k], divided by the total world population [N] times the size of the room [V]).*

Example: Histogram method

9

- Data: [0., 1., 2., 2., 2.5, 3., 3.6, 3., 4., 5., 4., 7., 9., 9.2], Total samples: $N = 14$

$$\text{Estimated Density} \approx \frac{K}{N \times V}$$



The Goldilocks Dilemma: Choosing the Right Volume (V)

10

- In theory, as you get infinite data, you can make the box infinitely small to get a perfect local measurement. In reality, you have a limited amount of data, creating a massive balancing act:
- **If the box (V) is too big:** You catch a lot of points (k is reliable), but your estimate is too blurry. It averages everything out and loses all the local details.
- **If the box (V) is too small:** The estimate becomes incredibly noisy and "ragged." Most boxes will have 0 points, and a few will have 1 point, creating sharp, unrealistic spikes.

Two Real-World Solutions

11

- To solve this dilemma, you can't let the box shrink too fast. Two famous methods handle this balance differently:

Method	What is Fixed?	What is Measured?	How it works
Parzen Window	Volume (V)	Count (k)	You decide on a fixed box size based on your total data size, then count how many points fall inside it.
k-Nearest Neighbor (k-NN)	Count (k)	Volume (V)	You decide on a fixed number of points you want to capture, and you blow up the box like a balloon until it hits exactly that many points.

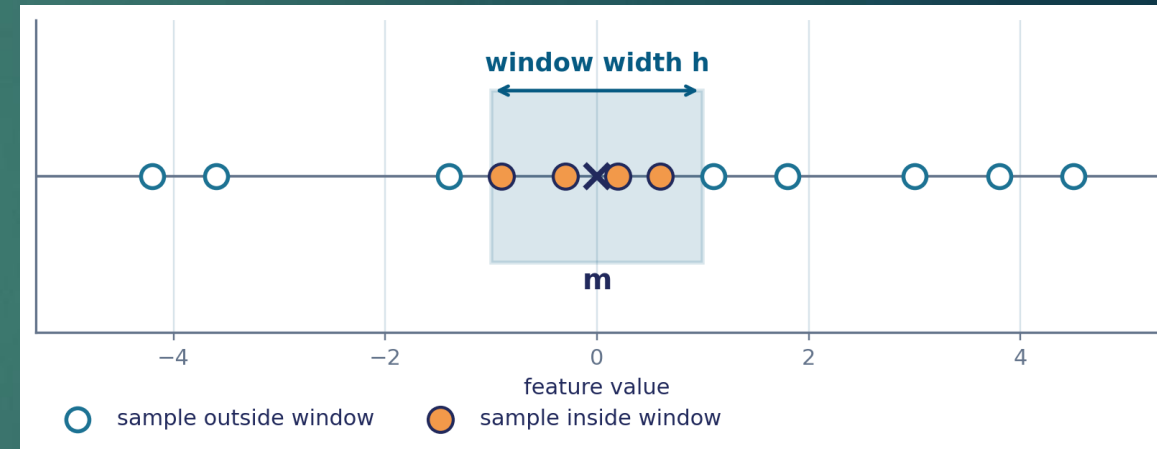
The Parzen window method

SECTION 5.1

THE BIG IDEA

Let the data draw its own density

- We want to know how likely different feature values are — the density $p(x)$ — but we don't know its formula.
- Parzen's trick: place a small "window" around any point m and count how many training samples fall inside it.
- More samples nearby $\rightarrow$ higher estimated density. Fewer samples nearby $\rightarrow$ lower density.
- Slide the window across the whole space and you get a full estimated density curve.

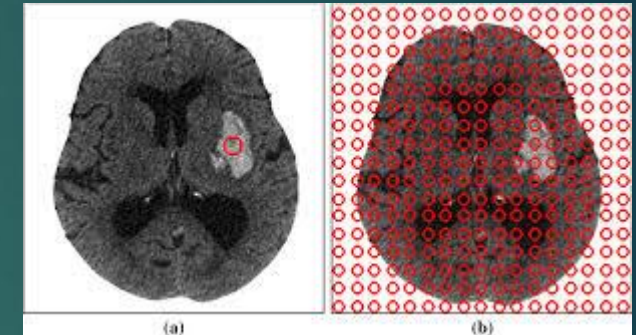


Only the orange samples inside the window count toward the density at "x".

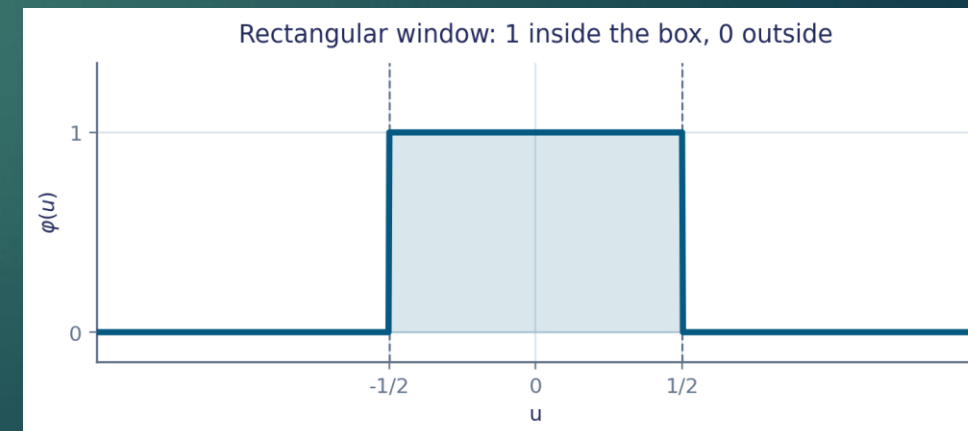
STEP 1 · DEFINE THE WINDOW

The window function $\varphi(u)$

- The simplest window is a box: it equals 1 for points close to the center, and 0 everywhere else.
- Think of it as a d-dimensional cube of edge length h, centered on “x”.
- A sample counts ($\phi = 1$) only if it lands inside that cube.



$$\varphi(u) = 1 \text{ if } |u| \leq \frac{1}{2}, \text{ else } 0$$

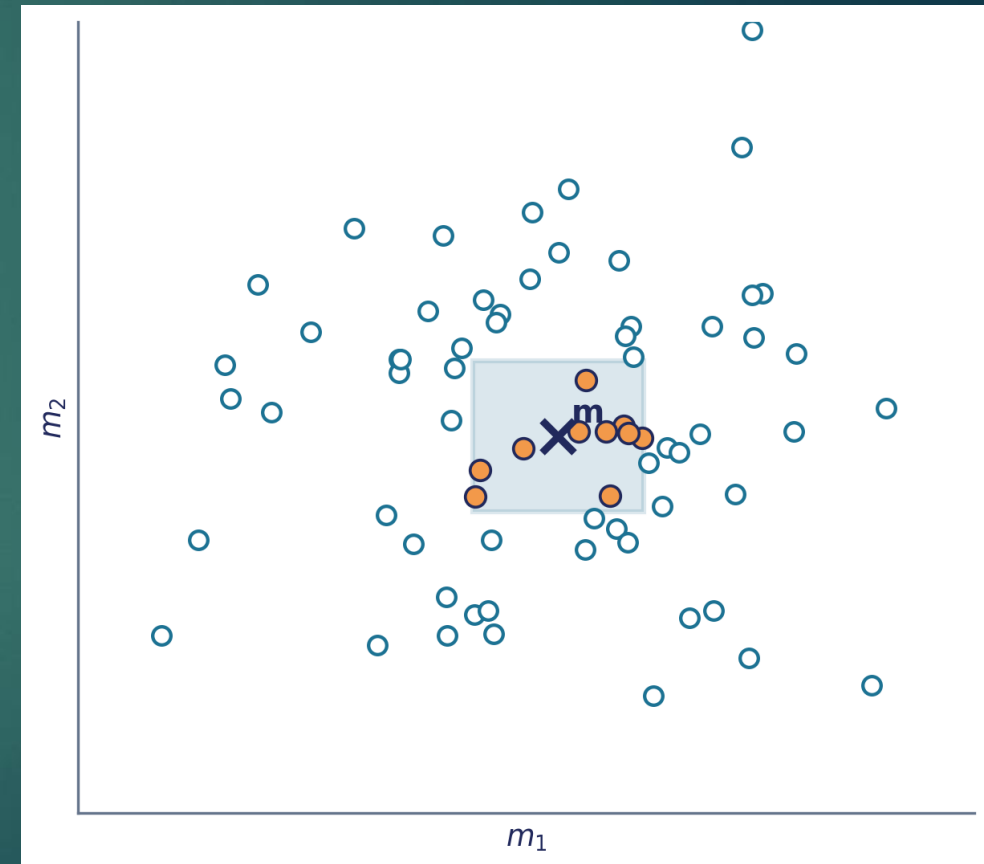


STEP 2 · COUNT WHAT'S INSIDE

How many samples fall in the window?

- For any point “x”, shift and scale the window to check every training sample.
- Sum up the ϕ values (each is 0 or 1) across all N samples.
- The result, $k_n(x)$, is simply the count of samples near “x”.

$$K(x) = \sum_{i=1}^N \phi \left(\frac{x - x_i}{h} \right)$$



Orange points are counted; the box is the neighborhood around “x”.

STEP 3 · TURN THE COUNT INTO A DENSITY

17

The Parzen density estimate

- Divide the count by N (sample size) and by the window's volume V_n — this normalizes it into a proper density.
- This is the one formula you actually compute for every point “ x ”.
- Each training sample contributes its own little "bump"; the estimate adds them all up and averages.

$$\hat{p}(x) = \frac{1}{N \times V_n} \sum_{i=1}^N \varphi \left(\frac{x - x_i}{h} \right)$$

$k_n(x)$ = count of samples near x $V_n = h_n^d$ = volume of the window

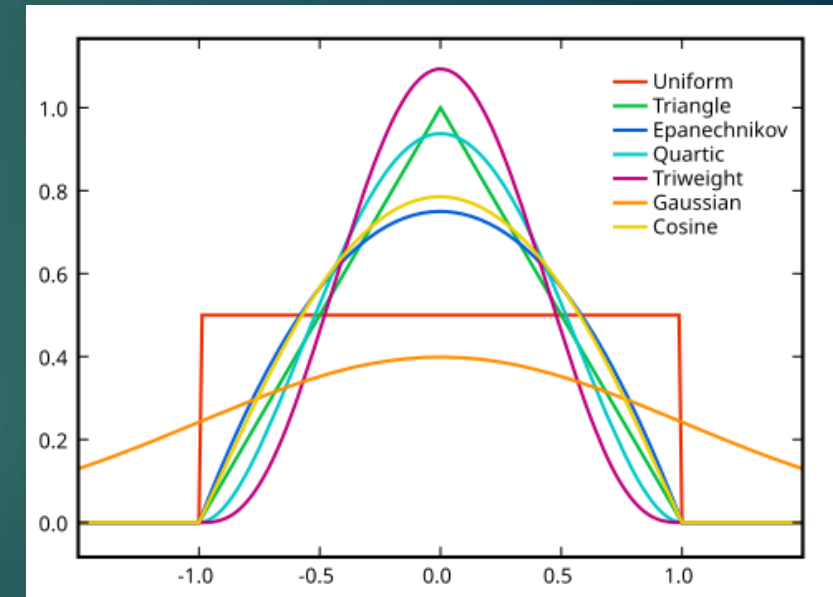
Type of Kernels

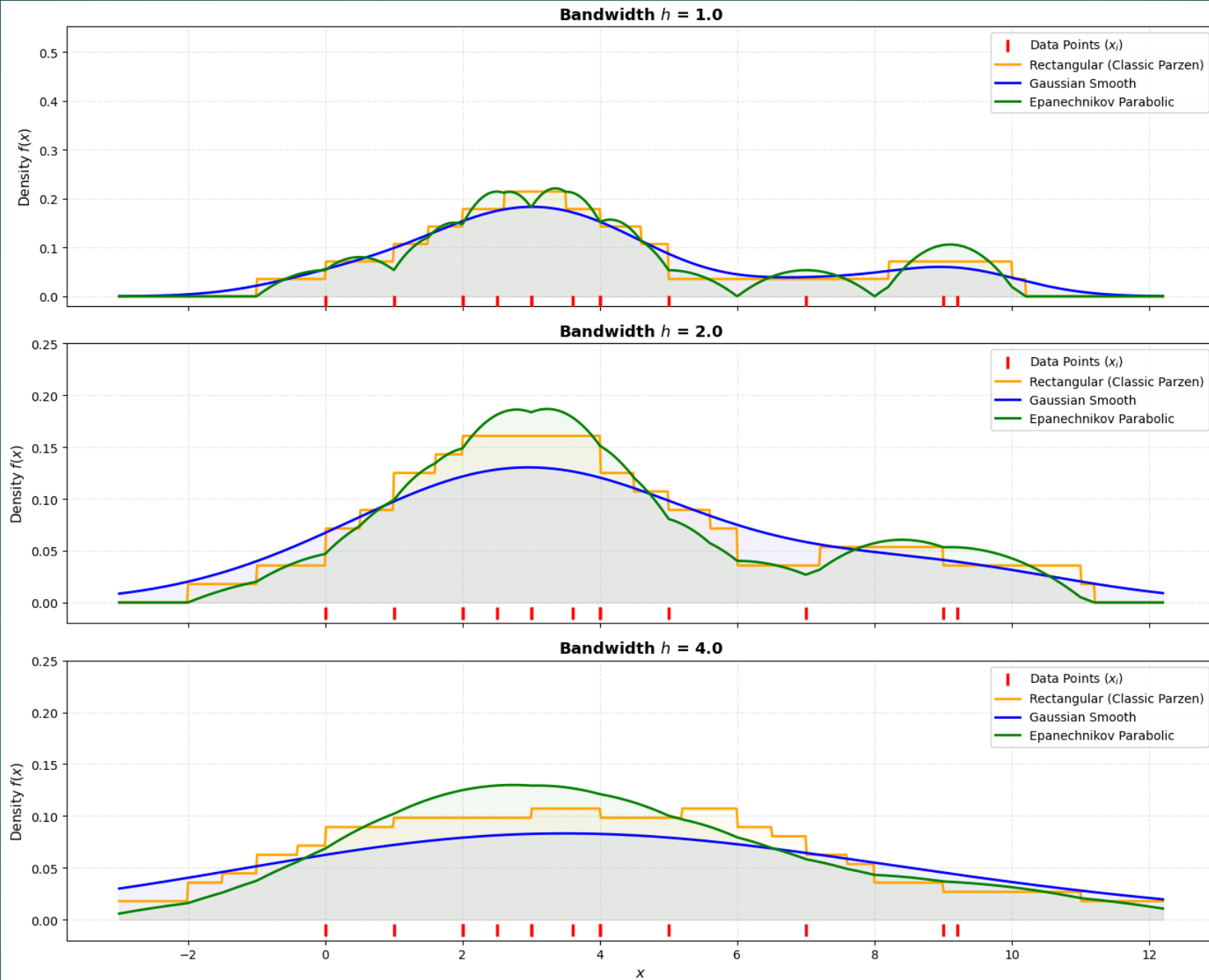
18

Kernel Functions, $K(u)$

Uniform ("rectangular window")	$K(u) = \frac{1}{2}$ Support: $ u \leq 1$	 "Boxcar function"
Triangular	$K(u) = (1 - u)$ Support: $ u \leq 1$	
Epanechnikov (parabolic)	$K(u) = \frac{3}{4}(1 - u^2)$ Support: $ u \leq 1$	
Quartic (biweight)	$K(u) = \frac{15}{16}(1 - u^2)^2$ Support: $ u \leq 1$	
Triweight	$K(u) = \frac{35}{32}(1 - u^2)^3$ Support: $ u \leq 1$	

Tricube	$K(u) = \frac{70}{81}(1 - u ^3)^3$ Support: $ u \leq 1$	
Gaussian	$K(u) = \frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}$	
Cosine	$K(u) = \frac{\pi}{4}\cos\left(\frac{\pi}{2}u\right)$ Support: $ u \leq 1$	
Logistic	$K(u) = \frac{1}{e^u + 2 + e^{-u}}$	
Sigmoid function	$K(u) = \frac{2}{\pi} \frac{1}{e^u + e^{-u}}$	
Silverman kernel ^[5]	$K(u) = \frac{1}{2}e^{-\frac{ u }{\sqrt{2}}} \cdot \sin\left(\frac{ u }{\sqrt{2}} + \frac{\pi}{4}\right)$	

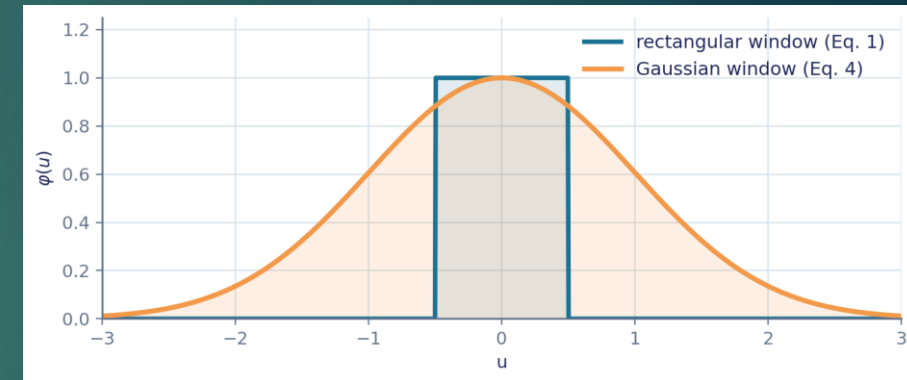




A SMOOTHER CHOICE

Swapping the box for a Gaussian bump

- The box window creates a ragged, blocky estimate — samples suddenly "switch on" at the edge.
- A Gaussian window instead fades smoothly from the center outward.
- It plugs into the exact same counting formula — just replace $\phi(u)$.



$$\phi(u) = \exp \left\{ -\frac{1}{2} \|u\|^2 \right\}$$

PUTTING IT TOGETHER

21

The final Gaussian Parzen estimate

- Combine the Gaussian window, the density formula, and the bandwidth rule into one expression you can compute directly from the samples:

$$\hat{p}(x) = \frac{1}{\sqrt{N} \times (2\pi)^{\frac{d}{2}} \times h^d} \sum_{i=1}^N \exp\left\{-\frac{N^{\frac{1}{d}}}{2 \times h^2} \|x - x_i\|^2\right\}$$

- N** number of training samples
- d** number of feature dimensions
- h** initial width (your design choice)
- x_i** the i-th training sample

You plug in “x”, loop over all N samples, and sum the Gaussian contributions — that's it.

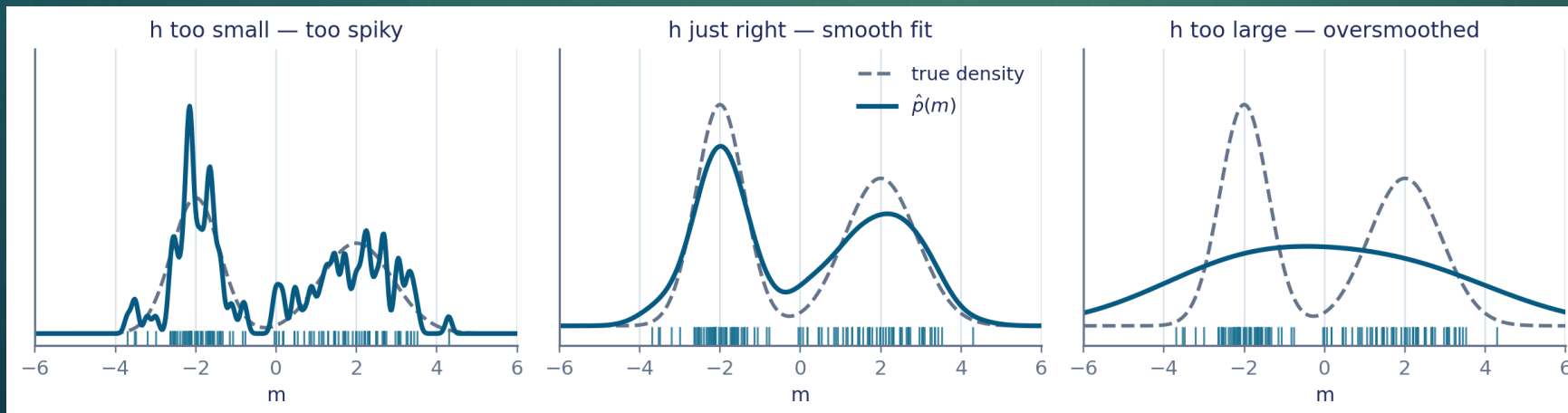
THE ONE SETTING THAT MATTERS

Choosing the bandwidth h

- h controls the window's width — the only parameter you pick.
 - Too small $\rightarrow$ estimate is spiky and noisy.
 - Too large $\rightarrow$ estimate is oversmoothed and blurs real structure.
 - In practice, h_n is shrunk as N grows, following:

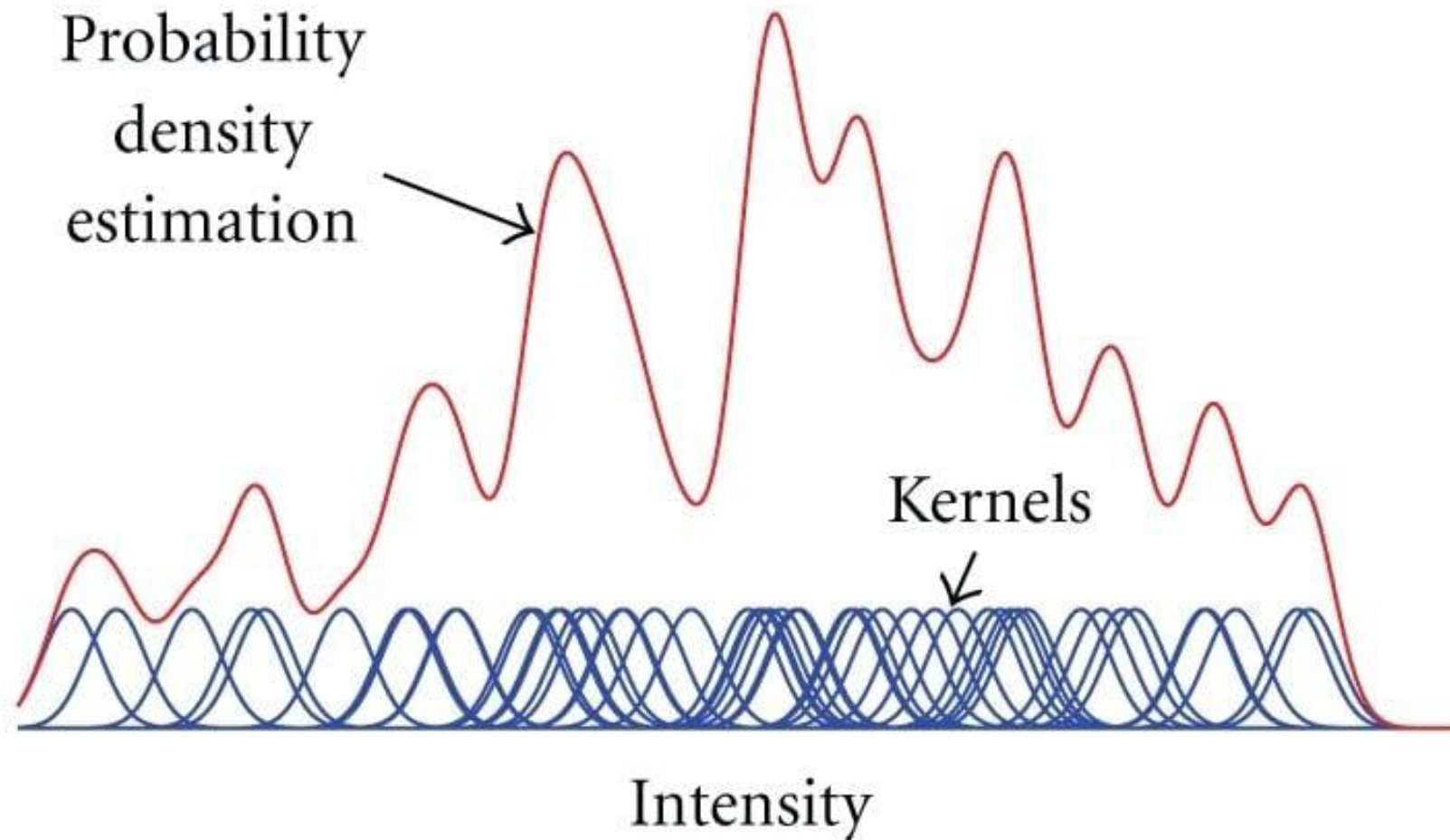
$$h_N = h \cdot N^{-\frac{1}{2d}}$$

h = initial width you choose
· d = number of feature dimensions



Gaussian Parzen estimate

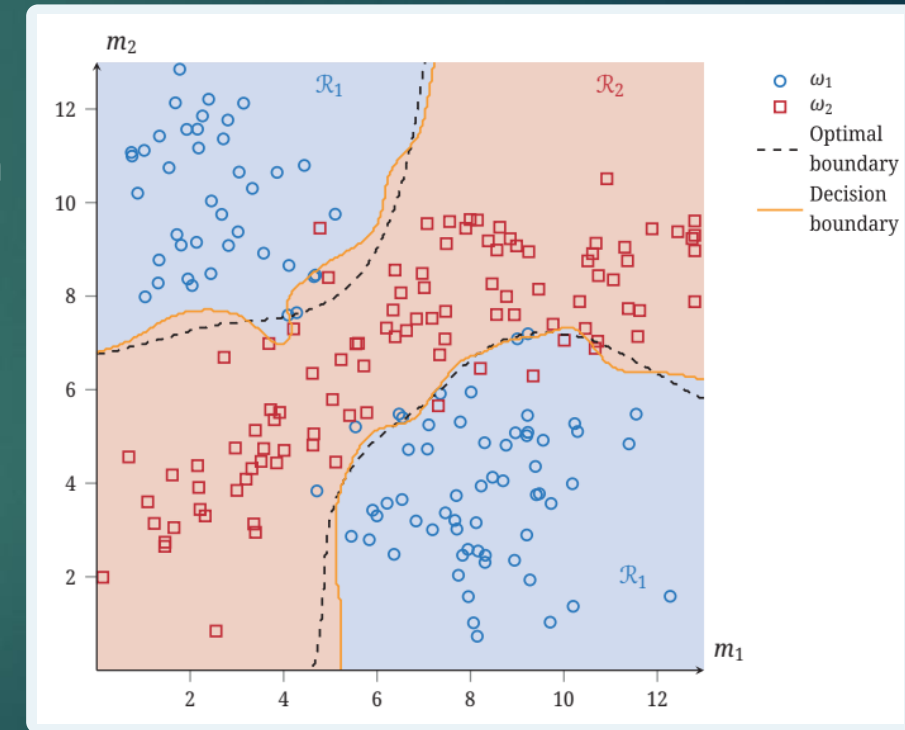
23



WHERE IT'S USED

Example: classification with Parzen windows

- Estimate one density per class: $p(x | \omega_1)$ and $p(x | \omega_2)$, each with the formula from the previous slide.
- Classify a new point “x” by picking the class with the higher estimated density (MAP rule).
- With a good bandwidth, the resulting decision boundary tracks the optimal Bayesian boundary closely.
- In the textbook example ($N = 100$ per class, $h_n \approx 0.5$): training error $\approx 6\%$, test error $\approx 6.5\text{--}7\%$, versus the 6.16% optimal.



Three traits of the Parzen window method

1

Universal

No prior assumption about the density shape needed — even multi-modal distributions can be estimated.

2

One knob to tune

Convergence is guaranteed in theory, but the quality in practice depends heavily on the choice of h .

3

Fixed neighborhood size

For a given N , every point in space uses the same window width, regardless of how dense the data is there.

Core equations to remember

- Density estimate: $\hat{p}(x) = (1/NV_n) \sum \phi((x - x_i)/h_n)$
- Bandwidth rule: $h_n = h \cdot N^{(-1/2d)}$

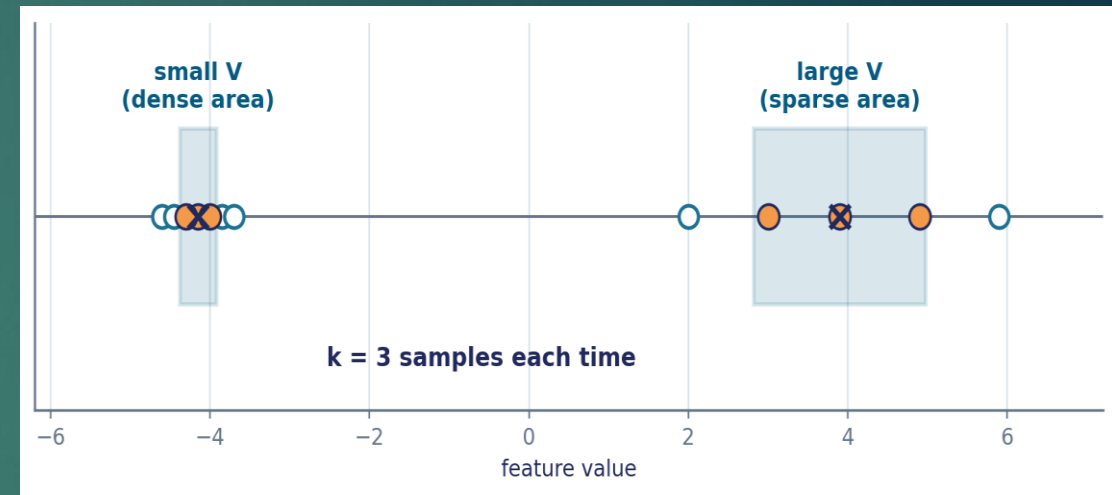
The k -nearest neighbor method

SECTION 5.2

THE BIG IDEA

Fix the count, let the window adapt

- Parzen window: pick a fixed window size, count how many samples land inside — the count k varies.
- k -nearest neighbor: pick a fixed count k , then grow the window until exactly k samples are inside — the volume V varies.
- Where data is dense, the window shrinks. Where data is sparse, the window grows.
- This makes the method automatically adapt to how crowded each region of the feature space is.



Same $k = 3$ neighbors, but a smaller window in the dense region and a larger one in the sparse region.

THE ONE FORMULA YOU NEED

The k-nearest neighbor density estimate

- *k is fixed in advance (your design choice).*
- *V is measured after inflating the neighborhood around x until it contains exactly k samples.*
- *N is the total number of training samples.*

$$\hat{p}(x) = \frac{k/N}{V} = \frac{k}{NV}$$

k = neighbors kept fixed

V = neighborhood volume (adapts to the data)

N = total sample count

Numerical Example

29

- Suppose we have one-dimensional Data $X = [2, 3, 7]$, and we need to estimate the distribution for this data using KNN with $K=2$.
- Since the Data is 1D, then Volume (V) = $2 * R$, where “ R ” is the Euclidian Distance.

At $X=2$

Neighbor	Distance
2	0
3	1
7	5

$$V = 2 * 1 = 2, \quad P(2) = \frac{2}{3 * 2}$$

At $X=3$

Neighbor	Distance
2	1
3	0
7	4

$$V = 2 * 1 = 2, \quad P(3) = \frac{2}{3 * 2}$$

At $X=7$

Neighbor	Distance
2	5
3	4
7	0

$$V = 2 * 4 = 8, \quad P(7) = \frac{2}{3 * 8}$$

Numerical Example with Test Point

30

- Suppose we have Data: [0. , 1. , 2. , 2. , 2.5, 3. , 3.6, 3. , 4. , 5. , 4. , 7. , 9. ,9.2], and we need to find probability for a test point $X = 5.8$ using KNN with $K=3$.
- First, we need to compute distance for this test point ($x=5.8$) to all data points, and then find the 3 nearest points

Data Point	0	1	2	2	2.5	3	3.6	3	4	5	6	7	9	9.2
Distance to $X=5.8$	5.8	4.8	3.8	3.8	3.3	2.8	2.2	2.8	1.8	0.8	0.2	1.2	3.2	3.4

$R = 1.2$, then $V = 2R = 2.4$

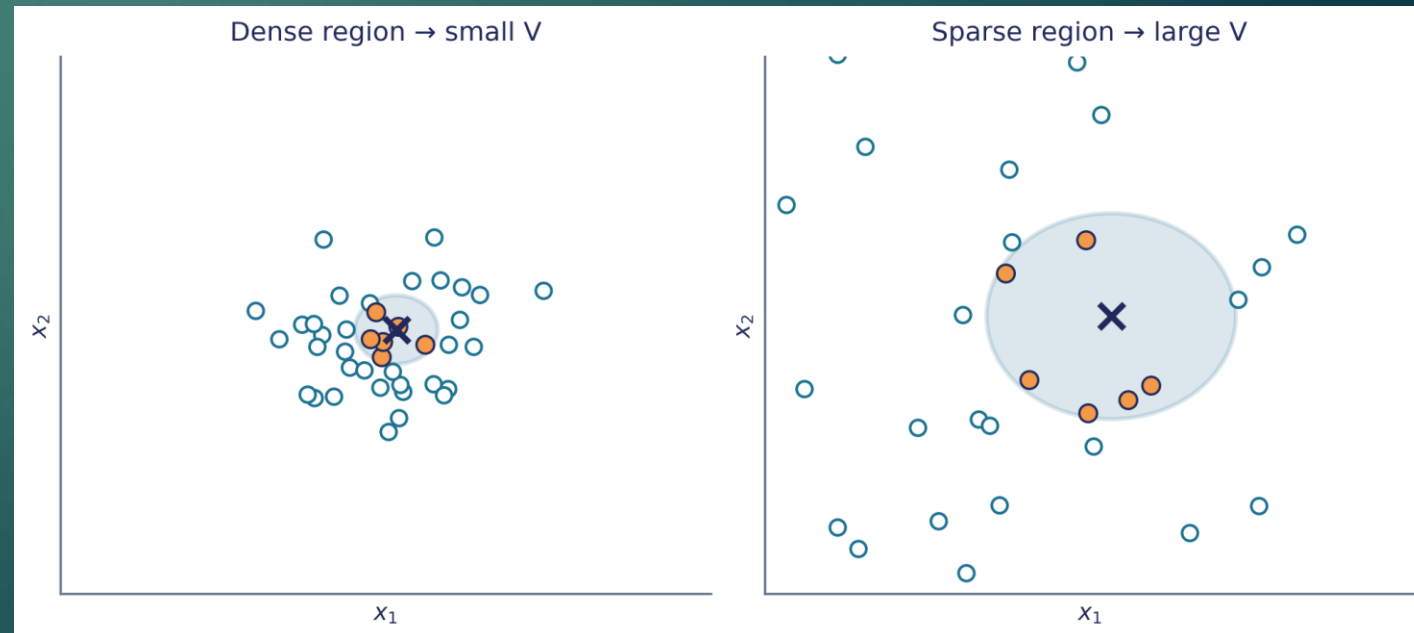
$$p(x = 5.8) = \frac{K}{N * V} = \frac{3}{14 * 2.4}$$

SEEING IT IN 2D

31

The neighborhood shrinks or grows on its own

- Think of the neighborhood as a balloon centered at x , inflating until it swallows exactly k points.
- In a crowded region the balloon stays small — it doesn't need to travel far to find k neighbors.
- In an empty region the balloon must expand a lot to find k neighbors, so V is large and the density estimate is low.



How to measure the Distance and then the Volume

32

- **Distance Metrics for k-NN**

- Different ways to measure "nearness" — and how they change the shape of neighborhoods.
- In k-NN, the choice of **distance metric** defines what it means for two points to be "close."
- Different metrics create different **neighborhood shapes**, which directly affects classification.

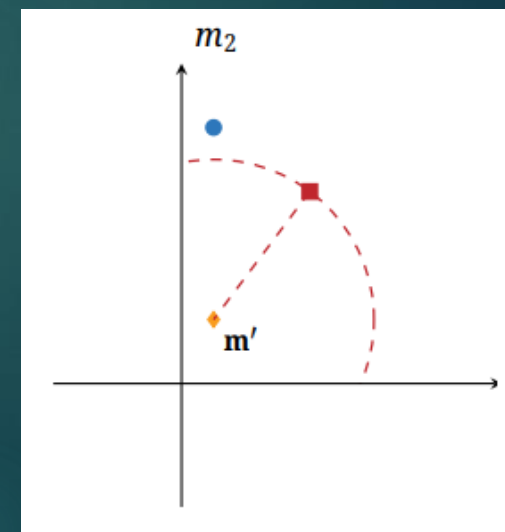
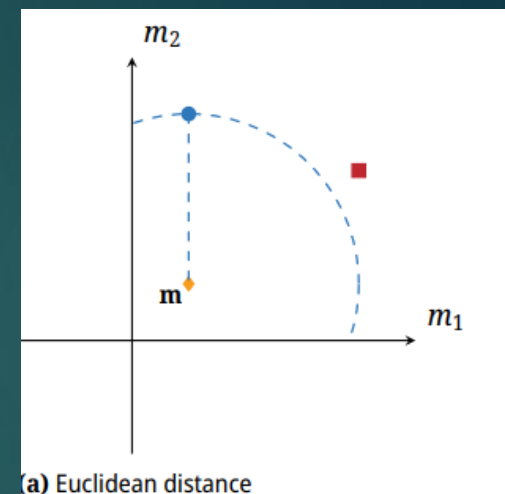
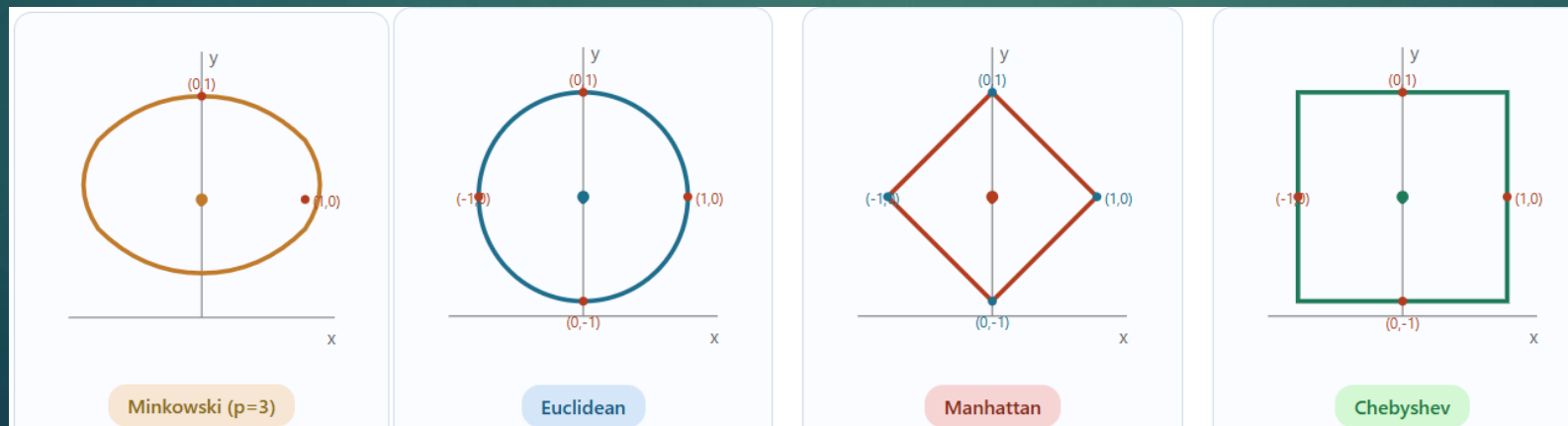
- **Key idea:**

- The metric determines the *shape* of the neighborhood around a test point.
- A spherical neighborhood (Euclidean) vs. a diamond-shaped one (Manhattan) vs. a square (Chebyshev) will include **different sets of neighbors**.

Detailed Metric Descriptions

33

- **Minkowski (Lp) “Generalized Form”**
 - $d = (\sum |x_{1,i} - x_{2,i}|^p)^{\frac{1}{p}}$
- **Manhattan (L1) City block:** Sum of absolute differences.
 - $d = |x_{1,1} - x_{2,1}| + |x_{1,2} - x_{2,2}| + |x_{1,3} - x_{2,3}| + \dots$
- **Euclidean (L2): “Most common”** Straight-line distance.
 - $d = \sqrt{|x_{1,1} - x_{2,1}|^2 + |x_{1,2} - x_{2,2}|^2 + |x_{1,3} - x_{2,3}|^2 + \dots}$



Calculating Volume

34

$$V = a \times R^d$$

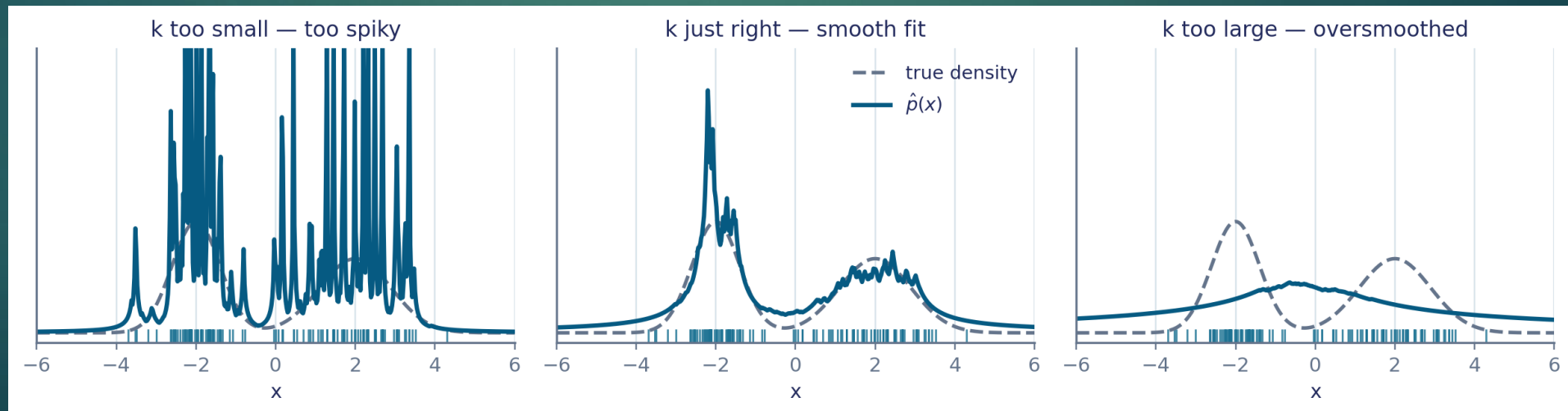
- Where:
 - R is the **Euclidean distance** from the test point m to its k -th nearest neighbor: $R = \sqrt{|x_{1,1} - x_{2,1}|^2 + |x_{1,2} - x_{2,2}|^2 + |x_{1,3} - x_{2,3}|^2 + \dots}$
 - “ a ” is the volume of a d -dimensional unit sphere ($a = 2$ for 1D, $a = \pi$ for 2D, $a = \frac{4}{3}\pi$ for 3D).
 - “ d ” is the dimensionality of the feature space.

THE ONE SETTING THAT MATTERS

35

Choosing k

- k controls how many neighbors define the local neighborhood — the only parameter you pick.
 - Too small → estimate is spiky and noisy.
 - Too large → estimate is oversmoothed and blurs real structure.
- A common practical rule of thumb: $K \approx \sqrt{N}$

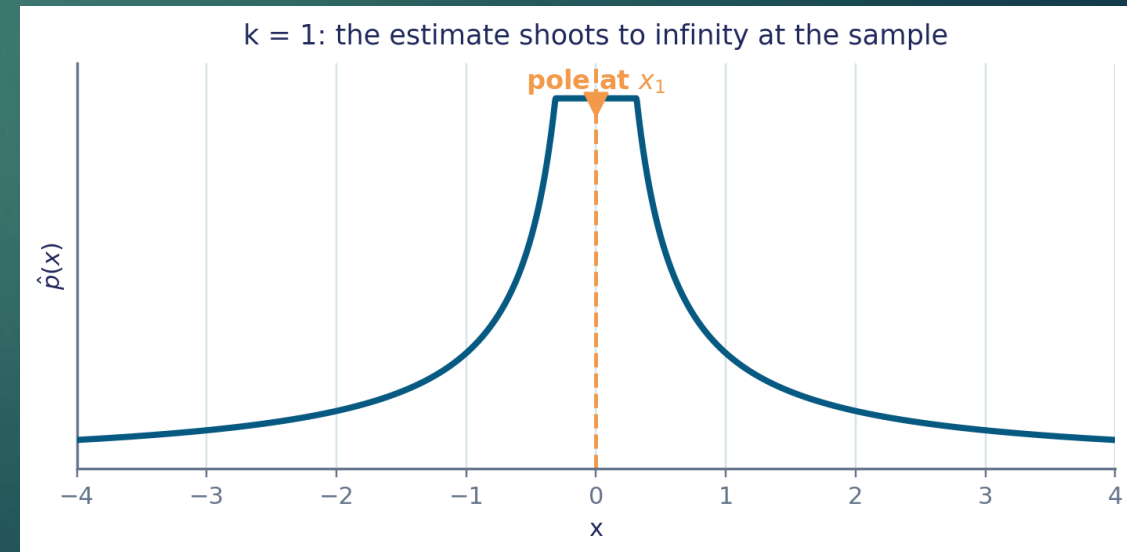


A QUIRK WORTH KNOWING

$k = 1$ makes the estimate blow up

- If $k = 1$, the neighborhood only needs to reach the single closest sample.
- Right at that sample, the neighborhood volume shrinks to zero — and dividing by zero sends the estimate to infinity.
- For a single sample x_1 in 1D with a spherical neighborhood:

$$\hat{p}(x) = \frac{1}{\alpha \|x - x_1\|}$$



Parzen window vs. k-nearest neighbor

37

	Parzen window	k-nearest neighbor
What's fixed	Window volume V	Neighbor count k
What varies	Count k found inside	Volume V needed
Smoothness	Smooth (with a smooth window)	Continuous, but not smooth
Dense regions	Same V everywhere	Small V automatically
Sparse regions	Same V everywhere	Large V automatically
Core Equation	$\hat{p}(x) = \frac{1}{N \times V} \sum_{i=1}^N \varphi \left(\frac{x - x_i}{h} \right)$	$\hat{p}(x) = \frac{K}{N \times V}$

- Both estimate $p(x) \approx (\text{mass in a neighborhood}) / (\text{volume of the neighborhood})$ — they just disagree on what stays fixed.

k-Nearest Neighbor Classification

SECTION 5.3

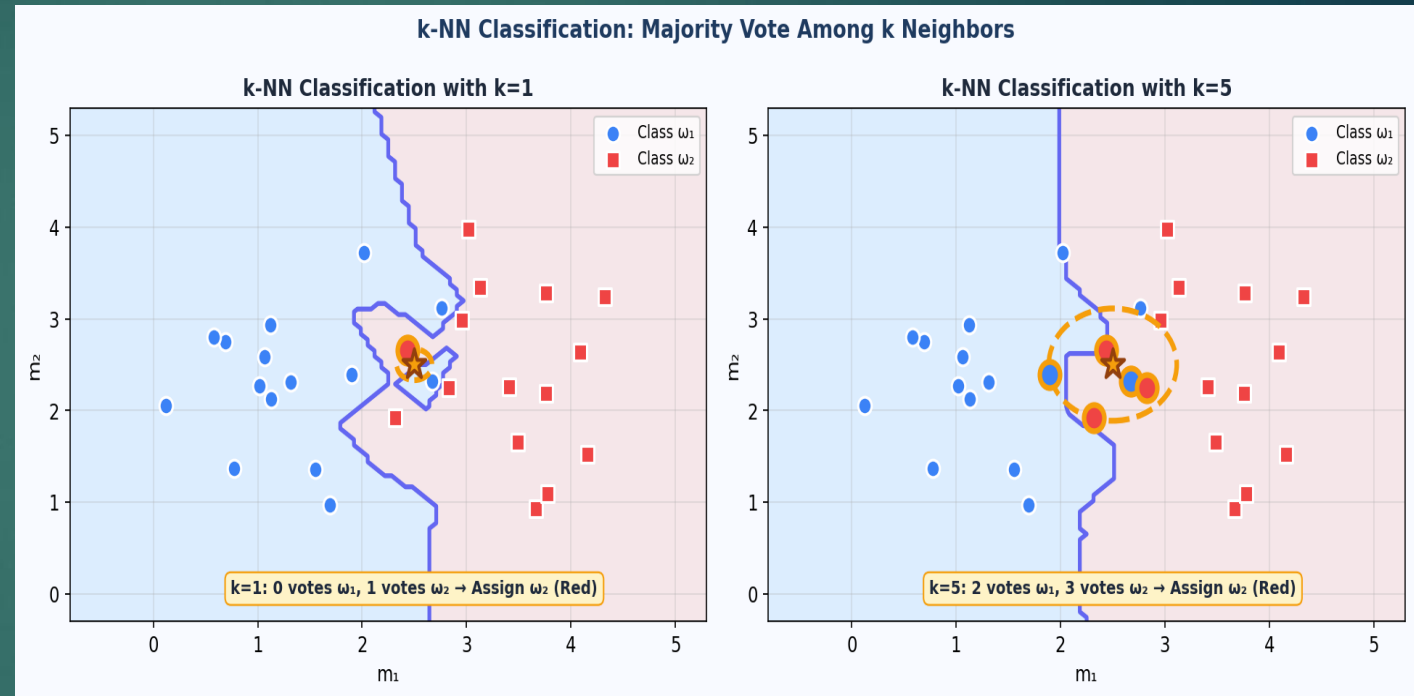
How k-NN Classification Works

39

$$\hat{p}(x, \omega_i) = \frac{K_i}{N * V}$$

Using Bayes Law:

$$\hat{p}(\omega_i|x) = \frac{K_i}{K}$$

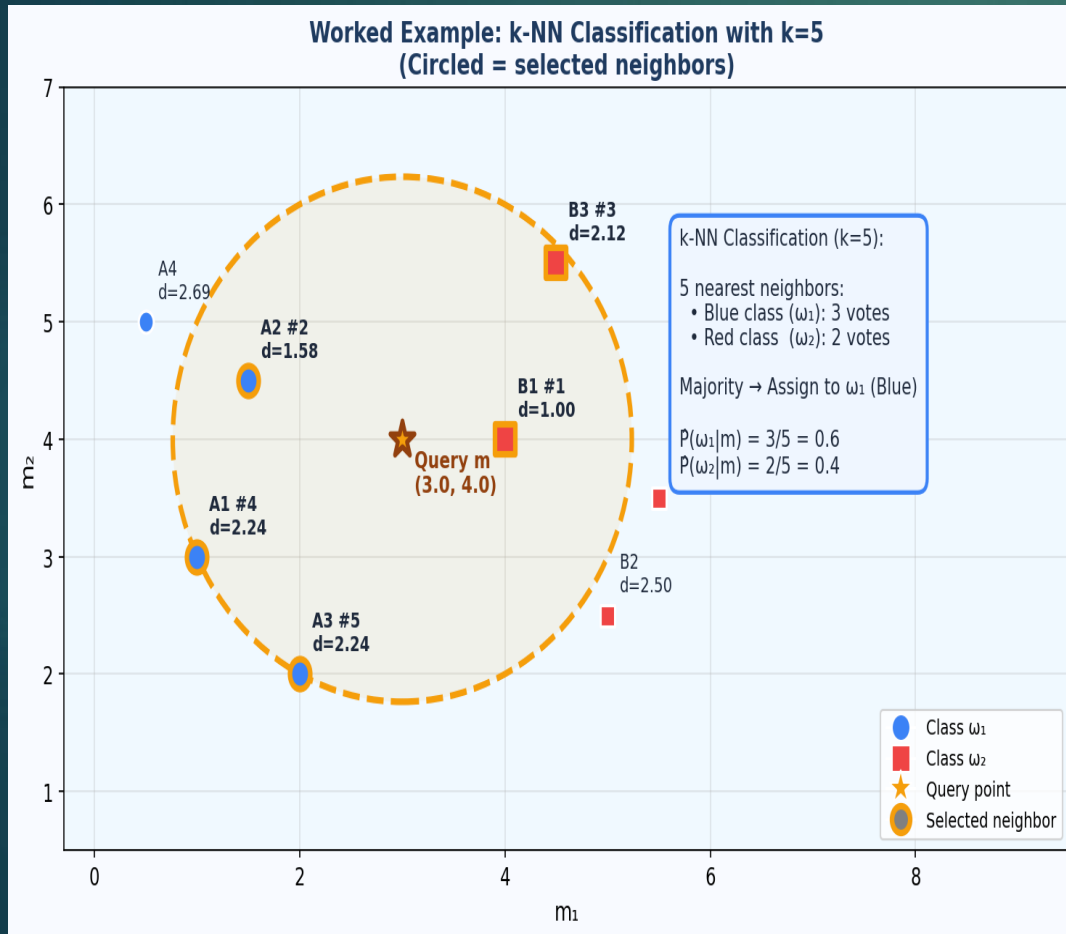


Algorithm:

- ① Choose k (e.g. k = 5)
- ② Find the k training points nearest to the unknown m
- ③ Count votes: how many belong to each class?
- ④ Assign m to the class with the most votes
- ⑤ Ties → pick randomly or by prior probability

Worked Example

40



Step 1: Setup

8 training points (4 blue ω_1 , 4 red ω_2)

Query: $m = (3.0, 4.0)$, $k = 5$

Step 2: Distances

A1:3.16 A2:2.06 A3:2.24 A4:3.20

B1:1.41 B2:2.50 B3:2.50 B4:2.69

Step 3: 5 Nearest

B1(ω_2 ,1.41), A2(ω_1 ,2.06), A3(ω_1 ,2.24)

B2(ω_2 ,2.50), B3(ω_2 ,2.50)

Step 4: Count Votes

ω_1 (blue): 2 votes ω_2 (red): 3 votes

Step 5: Decision

$P(\omega_1|m) = 2/5 = 0.4$

$P(\omega_2|m) = 3/5 = 0.6 \rightarrow$ Assign "X" to ω_2 (Red class)!

k=1: Decision Boundaries = Voronoi Cells

41

Nearest Neighbor Rule (k=1)

Assign the unknown point to the same class as its single closest training sample.

Decision Boundary

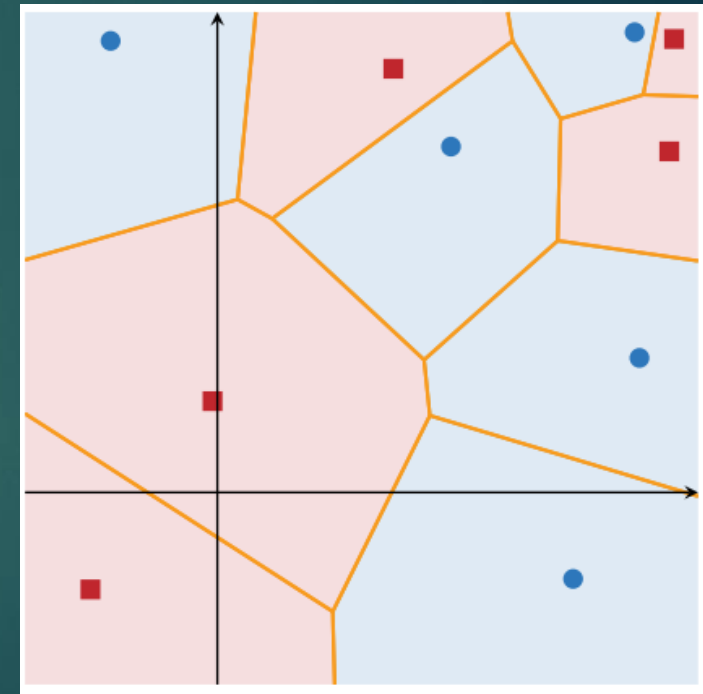
Each region in the feature space 'belongs' to the closest training point → Voronoi cells.

The Only Free Parameter

The distance metric (or scaling of axes). Changing scales changes which neighbor is 'nearest'.

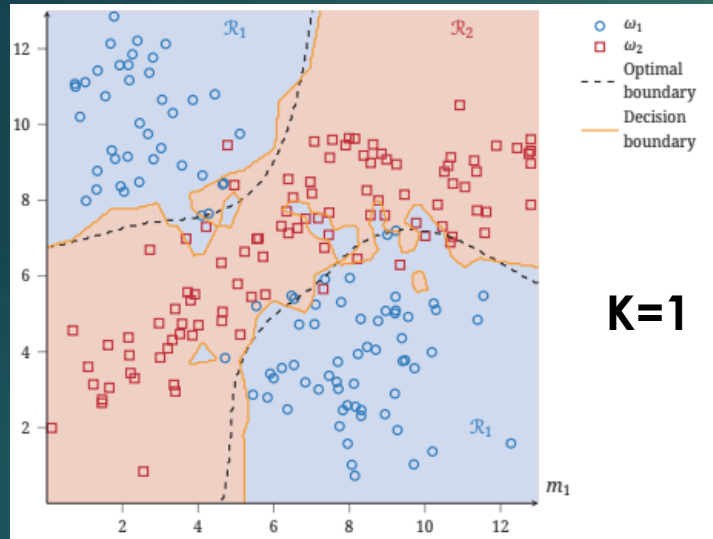
Limitation of k=1

Very rugged, sensitive boundaries. A single outlier creates an island.
→ Use larger k!

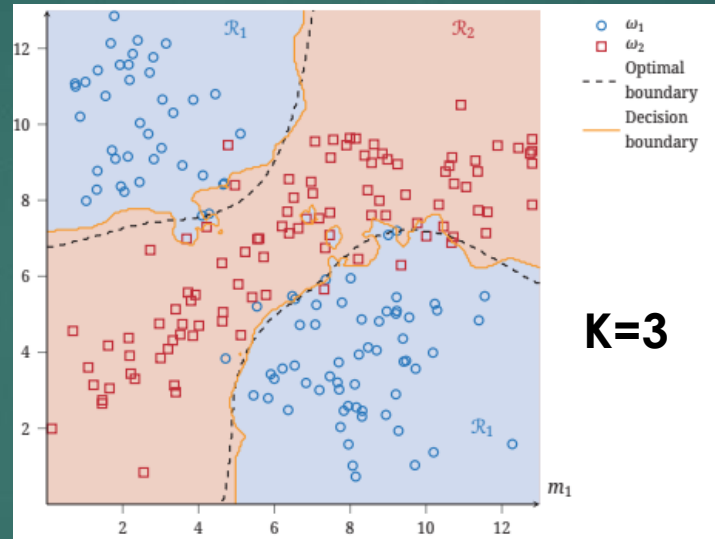


Effect of k on the Decision Boundary

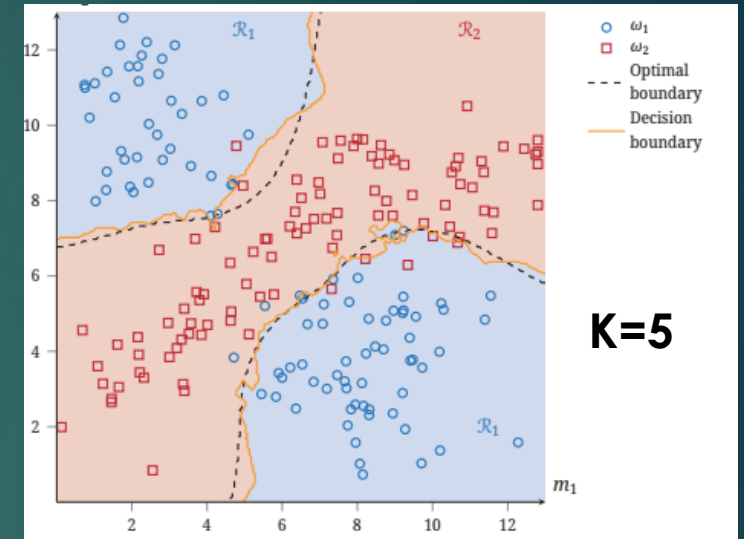
42



K=1



K=3

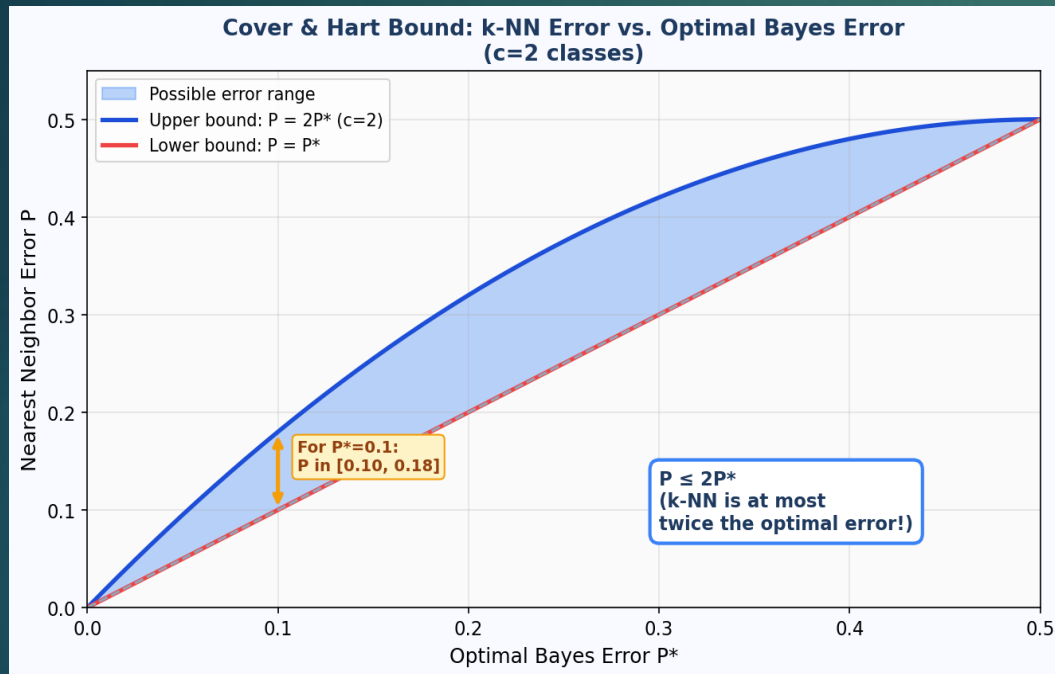


K=5

k value	Training Error	Test Error	Asymptotic (∞ samples)	Boundary Quality
k = 1	0%	10%	$\approx 9.4\%$	Rugged, complex
k = 3	4.5%	8%	$\approx 7.5\%$	Better, smoother
k = 5	7%	6%	$\approx 7.1\%$	Smooth, near-optimal

How Good Is k-NN? — The Cover & Hart Bound (1967)

43



Important caveat: This bound holds only asymptotically ($N \rightarrow \infty$). For finite N , there is **NO** guaranteed bound — convergence can be slow.

P^* = Optimal Bayes Error

The absolute best any classifier can do. Even a perfect model makes this many mistakes.

P = k-NN Error

Asymptotic error of the 1-NN rule as number of training samples $N \rightarrow \infty$.

The Bound: $P \leq 2P^*$

Nearest neighbor is at most **TWICE** as bad as the theoretically best possible classifier!

What This Means

Half the 'information' you could possibly extract from the data is captured by just looking at the single nearest sample.

Real-world Applications

Parzen Window (KDE)

45

- Brain Tissue Segmentation
- Automated analysis of 3D structural MRI scans isolates **Gray Matter (GM)**, **White Matter (WM)**, and **CSF** to track biomarkers for neurodegenerative diseases like Alzheimer's.
- Parzen Window Mechanism
- Gaussian window functions are applied to voxel intensities (x) across the population, resolving tissue overlap into three distinct, smooth peaks.



KNN Density Estimation

46

- **Cybersecurity (Network Anomaly & Intrusion Detection)**
- **The Real-World Context**
 - Corporate network routers generate massive volumes of continuous telemetry data, monitoring variables like *packet transfer rates*, *connection durations*, *protocol counts*, and *failed login attempts*.
 - Normal user operations form highly dense clusters in this multidimensional feature space, representing repetitive, predictable business activities.

